

INFO5995 Assignment 2 Report

Weihaio Cheng, Yucheng Zhao, Xu He, Yibo Tu, and Yuchen Zhou

1 APK Decompilation and Network Code Identification

This report analyzes the provided Android APK to identify insecure network communication. APK decompilation is necessary because Android apps are distributed in compiled form, and reverse engineering allows us to inspect the manifest and network-related code. I used JADX/JADX-GUI to inspect `AndroidManifest.xml` and `MainActivity.java`, where I found the `INTERNET` permission, `android:usesCleartextTraffic="true"`, `WebView` loading `http://www.example.com`, and `sslErrorHandler.proceed()`, which provide the main evidence for the later vulnerability analysis.

2 System and Threat Model

This analysis is based on static inspection of the decompiled code, without runtime execution.

The application initializes a `WebView` and automatically loads a remote web page from `http://www.example.com`, acting as a simple web content viewer that depends on external network resources.

During execution, the application communicates with a remote server to retrieve web content. In this process, data such as requested URLs, HTTP request metadata (e.g., headers), and server responses (e.g., HTML and JavaScript) may be transmitted. Depending on the web content, user input and session-related data may also be exchanged.

The system consists of three main components: the Android application (client), the remote web server, and the network connecting them. The network is considered untrusted and forms a trust boundary, as it cannot be assumed to provide secure communication.

We consider a threat model where an on-path attacker (e.g., on the same public Wi-Fi network) is able to intercept, modify, or inject traffic between the application and the server.

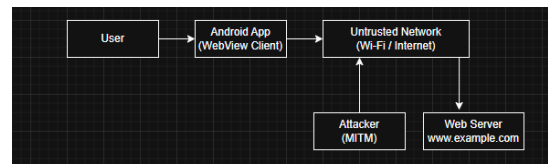


Figure 1: System and Threat Model

3 Insecure Transport Path

Static analysis shows that the provided APK contains an insecure transport issue. The app permits cleartext traffic and loads a web resource over HTTP, meaning that encrypted transport is not enforced for this communication path. This creates a network channel that could be exposed to man-in-the-middle (MITM) interception or tampering. Because HTTP provides neither encryption nor server authentication, an on-path attacker may be able to read or modify the traffic exchanged between the app and the remote endpoint. In addition, the code also contains an insecure SSL handling pattern in which `sslErrorHandler.proceed()` is called in `onReceivedSslError()`, indicating that certificate errors would be ignored if HTTPS content were loaded through the `WebView`.

4 Code and Configuration Evidence

In `AndroidManifest.xml`, the app sets `android:usesCleartextTraffic="true"`, which allows unencrypted HTTP traffic. In `MainActivity.java`, the `WebView` loads `http://www.example.com`. Together, these findings show that the app not only permits cleartext traffic in configuration, but also uses it in practice. The code also contains an insecure SSL handling pattern: `sslErrorHandler.proceed()` is called in `onReceivedSslError()`, indicating that certificate errors would be ignored if HTTPS content were loaded through the `WebView`.



Figure 2: Code Evidence 1

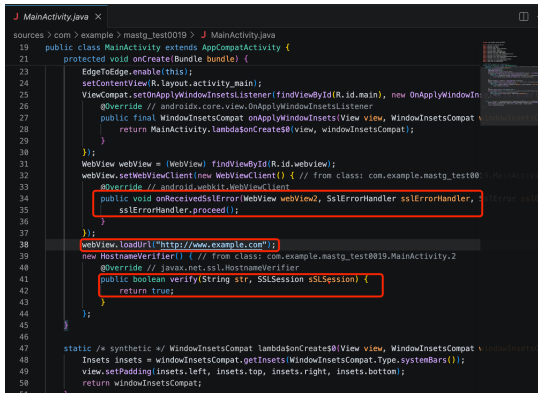


Figure 3: Code Evidence 2

5 Plain-Language Explanation

This application suffers from insecure network transmission and TLS verification issues, enabling attackers to launch man-in-the-middle attacks between the user and the server. The application not only allows plaintext HTTP traffic but also loads web page content directly via `http://`. Furthermore, when the `WebView` encounters an SSL certificate error, the program continues loading the page by calling `sslErrorHandler.proceed()` instead of terminating the connection. This means that in untrusted network environments such as public Wi-Fi or shared hotspots, attackers could intercept and tamper with the content loaded by the application.

In this scenario, attackers can read data transmitted over the network and modify the page content returned by the server, such as inserting false prompts, tampering with links, injecting malicious scripts, or redirecting users to phishing pages. Because the content seen by the user may have been replaced midway, this problem could lead to information leakage, the theft of sensitive data, and users being tricked into performing incorrect actions, representing a typical man-in-the-middle (MITM) attack risk.

6 Worst-Case Impact

In the worst-case scenario, an attacker could intercept and tamper with the content of web pages loaded by the application

in an untrusted network environment, thus launching a man-in-the-middle attack against the user. Because the application allows plaintext HTTP transmission and continues loading pages even when SSL certificate errors occur, attackers can not only read the communication content but also replace legitimate pages with forged ones, tricking users into entering sensitive information such as usernames and passwords.

Furthermore, attackers could insert false prompts, malicious links, or redirects into the page, misleading users into performing dangerous actions, potentially leading to account theft, session hijacking, or data breaches. Therefore, the risk of this vulnerability lies not only in information eavesdropping but also in the fact that attackers can simultaneously achieve content modification and user deception.

7 Mitigation Ideas

7.1 Concrete Mitigations

Enforce Encrypted Transport: Update the endpoint in `MainActivity.java` from `http://www.example.com` to `https://www.example.com` to ensure all `WebView` traffic is wrapped in a TLS tunnel.

Restrict Manifest Policy: In `AndroidManifest.xml`, change `android:usesCleartextTraffic` to `false`. This creates a platform-level restriction that prevents the app from initiating any non-HTTPS connections.

Define Network Security Configuration: Create a `network_security_config.xml` file to explicitly forbid cleartext traffic for the specific domain `example.com`, providing a robust, defense-in-depth security control.

7.2 Risk Reduction

These three mitigations provide a layered, defense-in-depth approach to neutralizing the MITM threat:

For Encrypted Transport (HTTPS): Transitioning to `https://` directly addresses the root cause by establishing a TLS tunnel. It restores the properties of Confidentiality and Authentication. An on-path attacker will only intercept unintelligible ciphertext, preventing sensitive data theft and malicious content injection.

For Restricting Manifest Policy: Setting `usesCleartextTraffic="false"` acts as a global, framework-level fail-safe. It prevents accidental use of insecure HTTP connections, eliminating risks caused by developer misconfiguration.

For Network Security Configuration: Defining domain-specific rules provides granular control. It ensures that even if the global manifest policy is inadvertently bypassed or misconfigured in the future, traffic specifically destined for `example.com` remains strictly enforced over TLS, effectively minimizing the application's attack surface.